

Mrp Rotation

Component Design Document

1 Description

MRP rotation attitude guidance. Superimposes a constant rate rotation on an input attitude reference frame, advancing the rotation by one control period each tick, and publishes the resulting reference attitude, rate, and acceleration. Wraps the MrpRotationAlgorithm C++ algorithm via its C shim.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 5
- **Number of Invokee Connectors** - 3
- **Number of Invoker Connectors** - 2
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - 2
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - 1
- **Number of Data Dependencies** - 1
- **Number of Packets** - *None*

3.2 Diagram

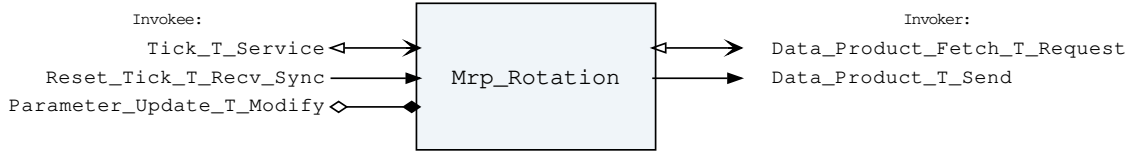


Figure 1: Mrp Rotation component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Mrp Rotation Invokee Connectors

Name	Kind	Type	Return_Type	Count
Tick_T_Service	service	Tick.T	Att_Ref.T	1
Reset_Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Parameter_Update_T_Modify	modify	Parameter_Update.T	-	1

Connector Descriptions:

- **Tick_T_Service** - Run the algorithm up to the current time and return the attitude reference it produces. The same reference is published as a data product.
- **Reset_Tick_T_Recv_Sync** - Restart the rotation from the configured initial attitude. Called on GNC state change so the rotation does not carry over from the previous state.
- **Parameter_Update_T_Modify** - The parameter update connector.

3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Mrp Rotation Invoker Connectors

Name	Kind	Type	Return_Type	Count
Data_Product_Fetch_T_Request	request	Data_Product_Fetch.T	Data_Product_Return.T	1
Data_Product_T_Send	send	Data_Product.T	-	1

Connector Descriptions:

- **Data_Product_Fetch_T_Request** - Fetch a data product item from the database.
- **Data_Product_T_Send** - The data product invoker connector

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 3: Mrp Rotation Set Id Bases Parameters

Name	Type
<code>Data_Product_Id_Base</code>	<code>Data_Product_Types.Data_Product_Id_Base</code>
<code>Parameter_Id_Base</code>	<code>Parameter_Types.Parameter_Id_Base</code>

Parameter Descriptions:

- **Data_Product_Id_Base** - The value at which the component's data product identifiers begin.
- **Parameter_Id_Base** - The value at which the component's parameter identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 4: Mrp Rotation Map Data Dependencies Parameters

Name	Type
<code>Base_Attitude_Reference_Id</code>	<code>Data_Product_Types.Data_Product_Id</code>
<code>Base_Attitude_Reference_Stale_Limit</code>	<code>Ada.Real_Time.Time_Span</code>

Parameter Descriptions:

- **Base_Attitude_Reference_Id** - The external data product ID used to fetch the value for the 'Base_Attitude_Reference' data dependency.
- **Base_Attitude_Reference_Stale_Limit** - The stale limit used for the 'Base_Attitude_Reference' data dependency. If the fetched data product is older than the given `Time_Span` then it is deemed stale and unfit for use by the component.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the MRP rotation algorithm with the control period and the default parameter values. The `init` subprogram requires the following parameters:

Table 5: Mrp Rotation Implementation Initialization Parameters

Name	Type	Default Value
Control_Period	Basic_Types.Positive_Short_Float	<i>None provided</i>

Parameter Descriptions:

- **Control_Period** - [s] Time between two algorithm updates. The rotation advances by this much per tick, so it must equal the rate at which the component is scheduled. Fixed by the assembly, so it is not a runtime parameter. The type keeps it greater than zero.

3.6 Commands

The Mrp Rotation component has no commands.

3.7 Parameters

Parameters for the Mrp Rotation component.

Table 6: Mrp Rotation Parameters

Local ID	Parameter Name	Type	Default Value
0x0000 (0)	Initial_Sigma_Rr0	Packed_F32x3.T	[0.0, 0.0, 0.0]
0x0001 (1)	Omega_Rr0_R	Packed_F32x3.T	[0.0, 0.0, 0.0]

Parameter Descriptions:

- **Initial_Sigma_Rr0** - [-] MRP attitude of the rotating frame R relative to R0 that the rotation starts from at initialization and on reset. Must be finite.
- **Omega_Rr0_R** - [rad/s] Constant angular velocity of R relative to R0 in R frame components. Must be finite.

3.8 Events

The Mrp Rotation component has no events.

3.9 Data Products

Data products for the Mrp Rotation component.

Table 7: Mrp Rotation Data Products

Local ID	Data Product Name	Type
0x0000 (0)	Attitude_Reference	Att_Ref.T

Data Product Descriptions:

- **Attitude Reference** - Attitude reference frame R rotating relative to R0. Attitude, rate, and acceleration of R relative to inertial.

3.10 Data Dependencies

Data dependencies for the Mrp Rotation component.

Table 8: Mrp Rotation Data Dependencies

Data Dependency Name	Type
Base_Attitude_Reference	Att_Ref.T

Data Dependency Descriptions:

- **Base_Attitude_Reference** - Attitude reference frame R0 that the rotation is superimposed on. Attitude, rate, and acceleration of R0 relative to inertial.

3.11 Packets

The Mrp Rotation component has no packets.

3.12 Faults

The Mrp Rotation component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Mrp_Rotation_Tests* Test Suite

This is a unit test suite for the Mrp Rotation component

Test Descriptions:

- **Test** - Run the algorithm through the rotation and reset sequence of the Python reference test to ensure the Ada to C to C++ integration is sound.
- **Test_Invalid_Parameter** - Ensure a staged configuration the algorithm would reject is refused at validation.
- **Test_Invalid_Data_Dependency** - Ensure a data dependency with the wrong identifier is treated as a wiring defect and fails the tick's assertion.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Att_Ref.T:

Table 9: Att_Ref Packed Record : 288 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sigma_Rn	Packed_F32x3.T	-	96	0	95
Omega_Rn_N	Packed_F32x3.T	-	96	96	191
Domega_Rn_N	Packed_F32x3.T	-	96	192	287

Field Descriptions:

- **Sigma_Rn** - No description provided.
- **Omega_Rn_N** - No description provided.
- **Domega_Rn_N** - No description provided.

Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 10: Data_Product Packed Record : 1096 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Data_Product_Header.T	-	72	0	71	-
Buffer	Data_Product_Types.Data_Product_Buffer_Type	-	1024	72	1095	Header.Buffer_Length

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

Data_Product_Fetch.T:

A packed record which holds information for a data product request.

Table 11: Data_Product_Fetch Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Data_Product_Types.Data_Product_Id	0 to 65535	16	0	15

Field Descriptions:

- **Id** - The data product identifier

Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 12: Data_Product_Header Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	48	63
Buffer_Length	Data_Product_ Types.Data_Product_ Buffer_Length_Type	0 to 128	8	64	71

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

Data_Product_Return.T:

This record holds data returned from a data product fetch request.

Table 13: Data_Product_Return Packed Record : 1104 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
The_ Status	Data_ Product_ Enums. Fetch_ Status.E	0 => Success 1 => Not_Available 2 => Id_Out_Of_Range	8	0	7	-
The_ Data_ Product	Data_ Product. T	-	1096	8	1103	-

Field Descriptions:

- **The_Status** - A status relating whether or not the data product fetch was successful or not.
- **The_Data_Product** - The data product item returned.

Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 14: Packed_F32x3 Packed Array : 96 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	3	96

Parameter.T:

Generic parameter packet for holding a generic parameter

Table 15: Parameter Packed Record : 792 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Parameter_Header.T	-	24	0	23	-
Buffer	Parameter_Types. Parameter_Buffer_Type	-	768	24	791	Header.Buffer_Length

Field Descriptions:

- **Header** - The parameter header
- **Buffer** - A buffer that contains the parameter type

Parameter_Header.T:

Generic parameter header for holding arbitrary parameters

Table 16: Parameter_Header Packed Record : 24 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Parameter_Types. Parameter_Id	0 to 65535	16	0	15
Buffer_Length	Parameter_Types. Parameter_Buffer_Length_Type	0 to 96	8	16	23

Field Descriptions:

- **Id** - The parameter identifier
- **Buffer_Length** - The number of bytes used in the parameter type buffer

Parameter_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 17: Parameter_Update Packed Record : 824 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Table_Id	Parameter_Types. Parameter_Table_Id	0 to 65535	16	0	15	-
Operation	Parameter_Enums. Parameter_Operation_Type.E	0 => Stage 1 => Update 2 => Fetch 3 => Validate	8	16	23	-

Status	Parameter_Enums. Parameter_Update_Status.E	0 => Success 1 => Id_Error 2 => Validation_Error 3 => Length_Error	8	24	31	-
Param	Parameter.T	-	792	32	823	-

Field Descriptions:

- **Table_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 18: Sys_Time Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces. Unsigned_16	0 to 65535	16	32	47

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of $1/(2^{16})$ sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 19: Tick Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Count	Interfaces. Unsigned_32	0 to 4294967295	32	48	79

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Data_Product_Enums.Fetch_Status.E:

This status denotes whether a data product fetch was successful.

Table 20: Fetch_Status Literals:

Name	Value	Description
Success	0	The data product was returned successfully.
Not_Available	1	No data product is yet available for the provided id.
Id_Out_Of_Range	2	The data product id was out of range.

Parameter_Enums.Parameter_Operation_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 21: Parameter_Operation_Type Literals:

Name	Value	Description
Stage	0	Stage the parameter.
Update	1	All parameters are staged, it is ok to update all parameters now.
Fetch	2	Fetch the parameter.
Validate	3	Validate all the parameters.

Parameter_Enums.Parameter_Update_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 22: Parameter_Update_Status Literals:

Name	Value	Description
Success	0	Parameter was successfully staged.
Id_Error	1	Parameter id was not valid.
Validation_Error	2	Parameter values were not successfully validated.
Length_Error	3	Parameter length was not correct.